

ОТЧЁТ ПО HW4 “MyDrive: многопользовательское файловое хранилище, протокол TCP”

ФИО: Александр Васюков

Группа: БПИ-235

Email: avvasiukov@edu.hse.ru

Цель работы

Разработать многопользовательское клиент-серверное приложение на Go поверх TCP, которое синхронизирует локальную директорию клиента с личной директорией пользователя на сервере, передаёт только изменившиеся данные, поддерживает несколько одновременных соединений и умеет работать как через пользовательский буфер, так и через DMA.

Декларация об использовании ИИ

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

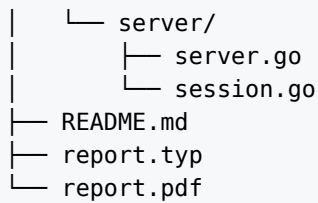
Использованные средства

- Язык программирования: Go 1.26.1
- Сетевые средства: стандартная библиотека Go, `net.Listener`, `net.Conn`, `bufio`
- Режим прямой передачи ядром: `(*net.TCPConn).ReadFrom(*os.File)`
- Операционная система проверки: macOS

Архитектура решения

Приложение состоит из клиентской и серверной частей, общего пакета протокола и пакета работы с файлами.

```
HW-04/
├── cmd/
│   ├── client/main.go
│   └── server/main.go
├── config/
│   ├── client_config.json
│   └── server_config.json
├── internal/
│   ├── client/client.go
│   ├── config/
│   │   ├── config_client.go
│   │   ├── config_helper.go
│   │   └── config_server.go
│   ├── files/files.go
│   ├── protocol/
│   │   ├── protocol.go
│   │   ├── reader.go
│   │   └── writer.go
```



Распределение обязанностей:

- клиент сканирует локальную директорию и вычисляет SHA-256;
- сервер строит разницу состояний и возвращает список файлов для передачи;
- одно управляющее соединение ведёт всю сессию синхронизации;
- отдельные соединения используются только для передачи содержимого файлов;
- сервер не ограничивает количество принимаемых соединений, а клиент ограничивает только число одновременно активных передач.

Пользовательский протокол поверх TCP

Управляющие сообщения передаются в следующем формате:

- первые 4 байта — длина сообщения в порядке байтов от старшего к младшему;
- следующий 1 байт — тип сообщения;
- далее идут полезные данные в формате JSON.

Такой формат выбран потому, что TCP передаёт непрерывный поток байт и не хранит границы сообщений.

Типы сообщений:

- ClientHello
- ServerHello
- ManifestRequest
- SyncPlan
- UploadRequest
- UploadAck
- SyncDone
- ErrorMessage

Порядок обмена:

1. Клиент открывает управляющее соединение и отправляет ClientHello.
2. Сервер создаёт идентификатор сессии и отвечает ServerHello.
3. Клиент отправляет одним сообщением ManifestRequest со списком файлов.
4. Сервер вычисляет разницу состояний и возвращает SyncPlan.
5. Для каждого нужного файла клиент открывает отдельное соединение передачи.
6. На файловом соединении клиент отправляет UploadRequest, затем содержимое файла.
7. Сервер принимает поток байт во временный файл, проверяет SHA-256, переименовывает файл и отвечает UploadAck.
8. После завершения всех передач клиент отправляет итоговое SyncDone.
9. Сервер удаляет устаревшие файлы и отправляет окончательное подтверждение.

Два режима передачи файлов

Передача через пользовательский буфер

В этом режиме клиент читает файл в пользовательский буфер фиксированного размера `buffer_size_bytes`, после чего записывает данные в TCP-соединение.

Передача через DMA

В режиме `dma` используется прямой путь `*os.File -> *net.TCPConn` через метод `ReadFrom`. На Unix-подобных системах этот путь переводит передачу в режим прямой работы ядра с файловым дескриптором и сокетом и убирает лишний проход через пользовательский буфер приложения.

Режим выбирается:

- через поле `transfer_mode` в конфиге;
- через команды `sync dma` и `sync buffered`;
- через команду `measure`, которая последовательно запускает оба режима на одном и том же наборе файлов.

Параллельная передача нескольких файлов

Клиент поддерживает до 32 одновременных соединений передачи. Значение задаётся в `max_connections`.

Схема работы:

- одно соединение всегда остаётся управляющим;
- на каждый файл открывается отдельное соединение передачи;
- активные передачи ограничиваются семафором размера `max_connections`;
- сервер принимает неограниченное число клиентских соединений и не накладывает искусственный предел.

Благодаря этому несколько больших файлов могут передаваться параллельно.

Конфигурационные файлы

Конфиг клиента:

```
{
  "client_id": "",
  "sync_dir": "../demo/client_data",
  "server_host": "127.0.0.1",
  "server_port": 9090,
  "max_connections": 8,
  "transfer_mode": "dma",
  "buffer_size_bytes": 1048576
}
```

Конфиг сервера:

```
{
  "host": "0.0.0.0",
  "port": 9090,
  "storage_root": "../demo/server_storage"
}
```

Запуск

Сервер:

```
cd HW-04
go run ./cmd/server -config ./config/server_config.json
```

Клиент:

```
cd HW-04
go run ./cmd/client -config ./config/client_config.json
```

Команды клиента:

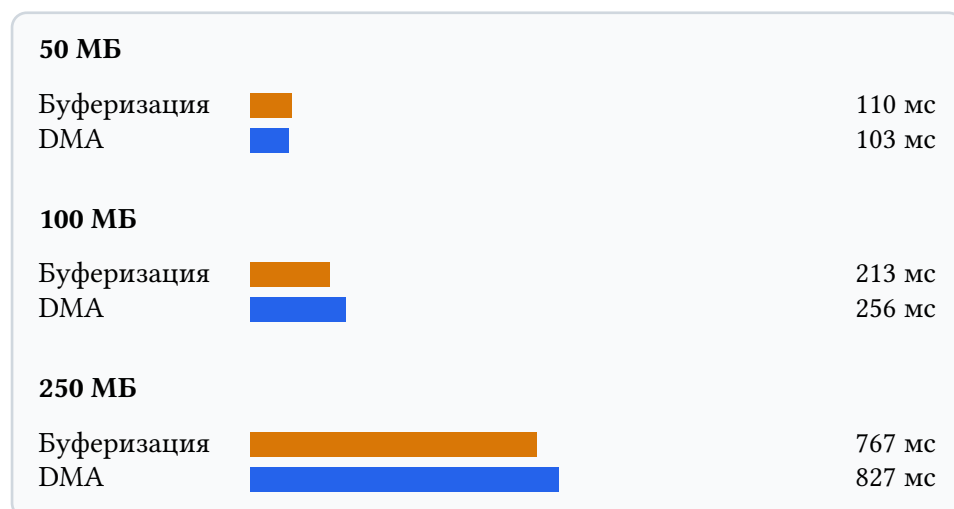
- sync
- sync dma
- sync buffered
- measure
- help
- exit

Замеры передачи 50 МБ, 100 МБ и 250 МБ

Для сравнения режимов передачи использовалась команда `measure`. На каждом размере выполнялись два принудительных прогона: сначала через пользовательский буфер, затем через DMA.

Размер файла	Буферизация, мс	DMA, мс	Буферизация, МБ/с	DMA, МБ/с
50 МБ	110	103	455.51	483.43
100 МБ	213	256	469.96	390.28
250 МБ	767	827	326.15	302.24

Графическое представление замеров



Наблюдения по TCP

В ходе обмена наблюдаются следующие свойства:

- для каждого раунда всегда есть одно управляющее TCP-соединение;
- для передачи файлов открываются дополнительные соединения;
- при `max_connections > 1` передача крупных файлов идёт параллельно;
- значения окна и максимального размера сегмента удобно наблюдать в SYN и SYN-ACK;
- команда `measure` позволяет сравнить влияние режима передачи при одинаковом наборе файлов.

Итог

Разработано полнофункциональное многопользовательское TCP-хранилище на Go.

Реализация включает:

- постоянный идентификатор клиента;
- передачу списка файлов с именем, размером и SHA-256;
- вычисление разницы состояний на сервере;
- загрузку только нужных файлов;
- удаление устаревших файлов;
- несколько одновременных соединений передачи;
- два режима пересылки данных: через пользовательский буфер и через DMA;
- встроенный механизм сравнительных замеров.